

UNIVERSITY OF NOTTINGHAM

School of Computer Sciences

COMP1234 - (2023-24)

Assignment 2- Workflow Satisfiability Problem

Symbolic Artificial Intelligence (COMP3008)

Submitted by:
Nderon Beqiri

Introduction

In this report, I present a solution to the workflow satisfiability problem using OR-Tools. My code aims to solve a scheduling problem involving various constraints specified in the document; authorization, binding of duty, separation of duty, at-most-k, and one-team.

While the functionality covers a wide range of scenarios, as each constraint was implemented, there might be specific cases where the solver does not always produce an optimal or feasible solution. This is primarily the case if there is only a unique solution in an incredibly difficult instance as the search of finding this sole solution would require too much time.

Overall this report provides the formulations used to implement my solver, how and if the solver works, and the strengths and weaknesses of said solver.

Formulations

assignStep[s] represents assigned user for step s

S represents set of Steps{1,2,..instance.n} U represents set of Users{1,2,…,instance.m}

1) Authorisation constraint: Scope is a subset of steps. If a step does not exist in scope, it cannot be assigned to the user
   $\exists scope \subseteq S$
   $\forall u \forall s(\neg(s \in scope) \rightarrow \neg(assignStep[s] = u))$

2) Binding of duty constraint: User assigned to step x must be same as step y
   $\forall s1 \forall s2(assignStep[s1] = assignStep[s2])$

3) Separation of duty constraint: User assigned to step x must be different to step y
   $\forall s1 \forall s2(assignStep[s1] != assignStep[s2])$

4) At most k constraint: Scope is a subset of steps. At most k users can do all steps in specified scope
   $\exists scope \subseteq S$
   $\forall u \forall s(|assignStep[s]| <= k)$

5) One team constraints: Scope is a subset of steps. Team is a subset of users. If a user is not in the team it cannot be assigned the step.
   $\exists scope \subseteq S, \exists team \subseteq U$
   $\forall s \forall u(\neg(u \in team) \rightarrow (assignStep[s] != u))$

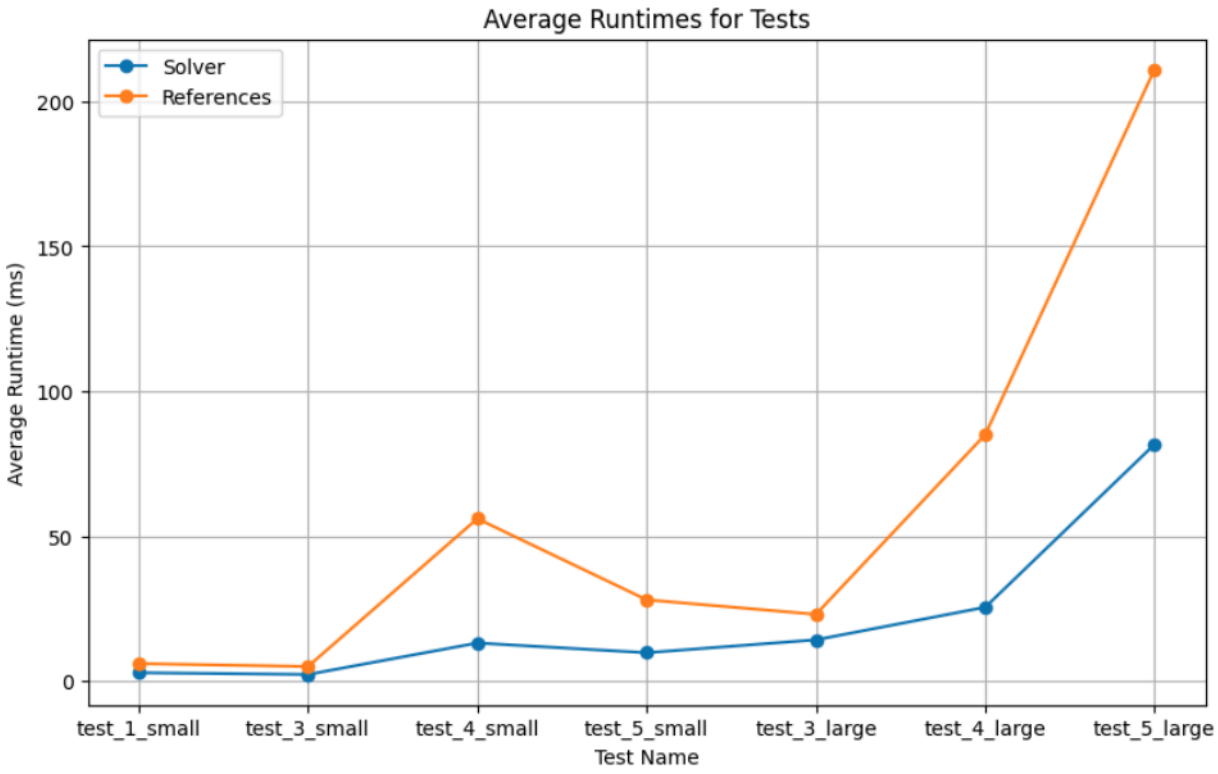
Implementation

The solver recognises whether there are no solutions, one solution or multiple solutions. The solver will go through all possible solutions and if there are none return unsat. If there is one solution, it will return sat, unique, and if the solver finds more than one solution, it will stop searching and return sat, not unique. It is not necessary for the solver to continue searching when the number of solutions exceeds 1, as we now know the solution is not unique, and looking for more solutions will simply waste time.

I was able to do this via implementing my own class, inspired by Stackoverflow, that is a custom solution callback using the Google OR-Tools library.
Its purpose is to count the number of solutions generated during the search process. Upon initialization, the class sets up a counter, to keep track of the number of solutions found. A method is invoked each time a solution is discovered. In this method, the solution count is incremented, and if it surpasses one, the search is halted. This design ensures that the solver terminates when it identifies more than one solution, effectively restricting the search to find whether a solution is unique.

Evaluation

I was able to run all tests apart from '4-constraint-hard.' All tests passed and would match given reference times and in some cases perform better. As additional functionality, I created a line graph to measure performance of the solver for comparison between reference times and my solver times for the first 7 test instances, and as shown below it highly outperformed the reference times.



Conclusion

Strengths

- Flexibility
  - The solver is capable of handling various types of constraints simultaneously.
- Extensibility
  - The code is structured in a modular way, allowing the easy addition of new constraints.
- Solution Counting
  - The inclusion of a custom solution callback allows for counting the number of solutions. This feature is beneficial when seeking a unique solution or when assessing the complexity of the problem.
- Performance
  - For the first 7 instances, it performs exceptionally well, each beating the reference times.

Weaknesses

- Search Strategy
  - The code uses a default search strategy provided by OR-Tools. Depending on the problem, a more specialized search strategy could be beneficial. Customizing the search strategy might enhance the solver's efficiency. Potential reason as to why '4-constraint-hard' was not performing well.

Possible Improvements

- Performance Tuning:
  - Experimenting with different search strategies could potentially improve the solver's performance on specific problem instances.

In conclusion, while the solver demonstrates versatility in handling diverse constraints, there is room for improvement in terms of efficiency for large instances.